មេរៀនទី១០

# EXCEPTION IN JAVA

# 1. WHAT IS AN EXCEPTION ?

**An Exception** is an **error or unexpected event** that occurs during program exception and interrupts the normal flow of the program.

Example situations:

+ Dividing by Zero
+ Accessing an array index that does not exist
+ Opening a file that does not exist

If exceptions are not dandled, the program **Crashes( terminates ).**

# 2. TYPES OF EXCEPTIONS IN JAVA



Java exceptions are divided into two main types:

| Type | Meaning | Example |
|---|---|---|
| Checked Exception | Checked at compile time | IOException, SQLException |
| Unchecked Exception | Happens during runtime | ArithmeticException, NullpointerException |

## 2.1 Checked Exception

Must be handled using try-catch or throws.

```
class Test {
        public static void main( String[] args ) throws IOException {
                FileReader f = new FileReader( "test.txt" );
        }
}
```

# 2. TYPES OF EXCEPTIONS IN JAVA

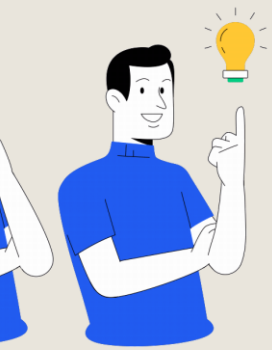


## 2.2 Unchecked Exception

Occurs during runtime.

Example:

```
class Text {
        public static void main(String[] args) {
                int  a = 10;
                int  b = 0;
                int  c = a  /  b;            //ArithmeticException
                System.out.println(c);
        }
}
```

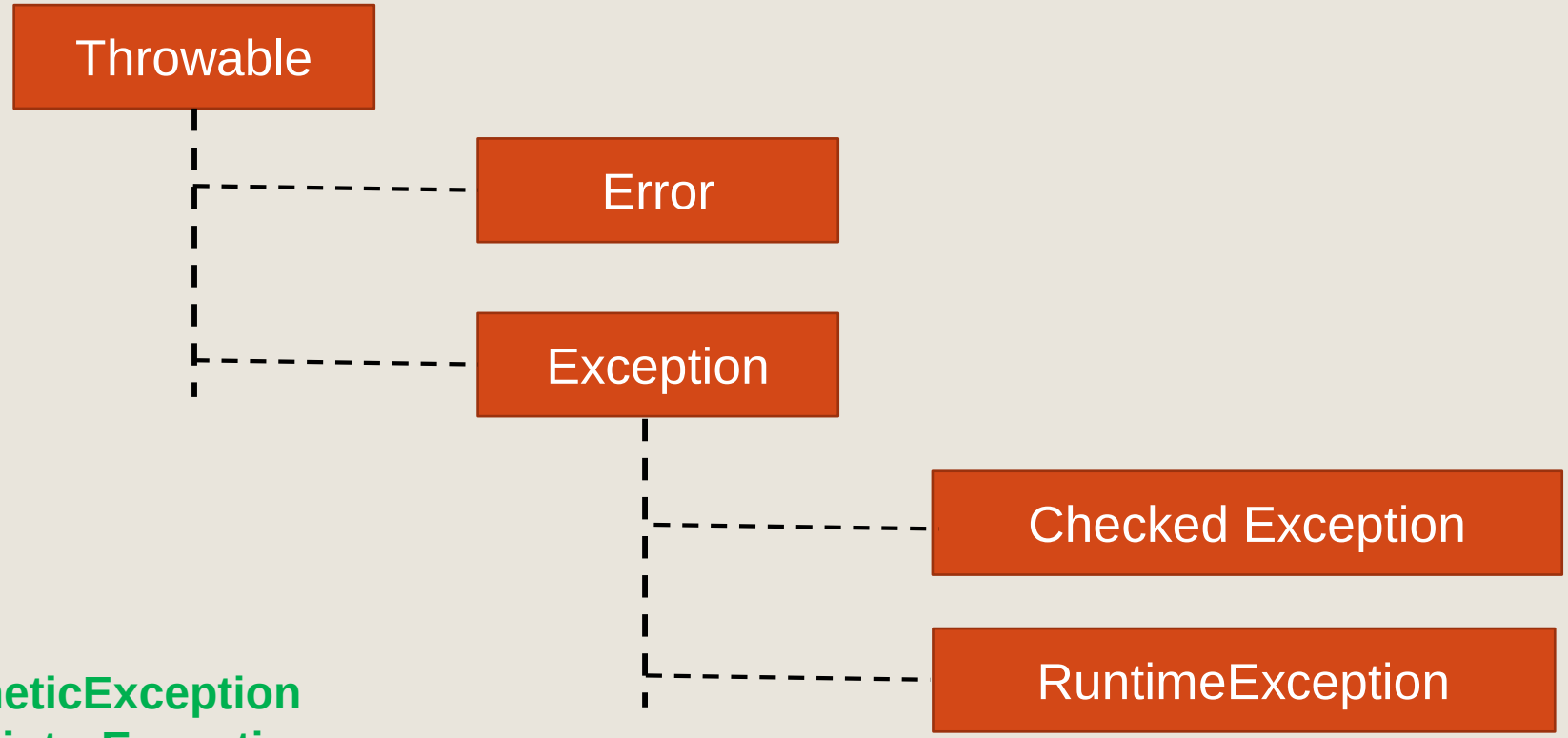


# 3. EXCEPTION HIERARCHY



All exceptions come from the main class: **Throwable**

**Structure:**




**Example:**

**+ ArithmeticException**
**+ NullpointerException**
**+ ArrayIndexOutOfBoundsException**
**+ IOException**

# 4. EXCEPTION HANDLING KEYWORDS IN JAVA



| Keyword | Meaning |
| --- | --- |
| try | Code that may cause exception |
| catch | Handle the exception |
| finally | Always executes |
| throw | Manually create exception |
| throws | Declare exception |

# 5. TRY-CATCH EXAMPLE



**Example:**

```
class Test {
    public static void main(String[] args) {
        try {
            int   a = 10;
            int   b = 0;
            int   c = a  /  b;
            System.out.println(c);
        }
```

# 5. TRY-CATCH EXAMPLE



```
            catch(ArithmeticException e ) {
                System.out.println("Cannot divide by zero");
            }
            System.out.println(Program continues…");
        }
    }
```

Output

Cannot divide by zero
Program continues…

# 6. MULTIPLE CATCH

**Example:**

```
class Test {
        public static void main(String [] args) {
                try {
                        int arr[] = {1,2,3};
                        System.out.println(arr[5]);
                }
                catch(ArrayIndexOutOfBoundsException e ) {
                        System.out.println("Array index error");
                }
                catch(Exception e ) { System.out.println("General error"); }
        }
}
```

# 7. FINALLY BLOCK



**Finally always exceptions whether an exception occurs or not.**

Example:

```
class Test {
        public static void main(String[] args){
                try { int  a = 5 / 0 ; }
                catch(ArithmeticException e) {
                        System.out.println(“Error occurred”);
                }
                finally {
                        System.out.println(“This always runs”);
                }
        }
}
```

Output

Error  occurred

This always runs

# 8. THROW EXAMPLE



**Used to manually create an exception**.

```
class Test {
        public static void main(String[] args) {
                int age  = 15;
                if  (age  <  18 ) {
                    Throw new ArithmeticException(“Not allowed to vote”);
                }
                System.out.println(“You can vote”);

        }
}
```

# 9. THROWS EXAMPLE



Used to manually create an exception.

```
class Test {
        static void readFile() Throws IOException {
                FileReader  f  = new FileReader(“data.txt”);
        }
        public static void main(String[] args )  Throws IOException {
                readFile();
        }
}
```

# 10. CUSTOM EXCEPTION



**You can create your own exception.**

```
class MyException extends Exception {
        MyException(String message){    supper(message); }
}
class Test {
        public static void main(String [] args) {
                try {  throw new MyException(“Custom Error occurred”); }
                catch(MyException e) {
                        System.out.println(e.getMessage());  }
        }
}
```

# 11. USER INPUT

```
class Test {
        public static void main(String args) {
                Scanner sc = new Scanner(System.in);
                try {     System.out.println(“Enter number:”);
                        int num = sc.nextInt();
                        int result = 100 / num ;
                        System.out.println(“Result  = “ +result);  }
                catch(ArithmeticException e ) {
                        System.out.println(“Cannot divide by zero”);  }
                catch(Exception e ) { System.out.println(“Invalid input”); }
        }
}
```

# 12. COMMON JAVA EXCEPTION



| Exception | Meaning |
|---|---|
| ArithmeticException | Divide by zero |
| NullPointerException | Using null object |
| ArrayIndexOutOfBoundsException | Invalid array index |
| NumerFormatException | Wrong number format |
| IOException | File input/output error |

# សូមអរគុណ